

CONSTITUCIÓN DE ARQUITECTURA DE SOFTWARE

Studio Dental — Clinical OS & Practice Management System

Documento: Norma Suprema e Inviolable de Ingeniería

Rol: Chief Software Architect / Principal Engineer

Versión: 3.0.0 (Definitiva y Consolidada)

Estado: **VIGENTE & MANDATORIO**

CAPÍTULO I: FILOSOFÍA, VISIÓN Y PRINCIPIOS DE DISEÑO

1.1 Visión del Proyecto

Studio Dental no es una simple interfaz de registro de citas. Es un **Sistema Operativo Clínico Odontológico (Clinical OS)** integral, modular y desacoplado, diseñado para operar en clínicas privadas y en el sector público de salud. Su objetivo central es garantizar la continuidad, inmutabilidad y trazabilidad clínica estricta de cada diagnóstico, optimizando la interacción financiera y visual entre odontólogo y paciente.

1.2 Filosofía de Desarrollo

- Precisión Médica sobre Velocidad Indiscriminada:** La integridad de los datos clínicos (anamnesis, odontogramas, fármacos, periodontogramas) jamás se comprometerá por optimizaciones prematuras. Cada evento es inmutable.
- Tolerancia Cero a la Deuda Técnica (Zero-Debt Policy):** Ningún código se integrará a producción si viola la separación de responsabilidades, genera acoplamiento entre módulos o carece de validación de esquemas.
- Resiliencia Operativa Offline-First:** Operatividad 100% garantizada en entornos sin conectividad mediante almacenamiento local híbrido normado (síncrono/asíncrono).

1.3 Principios de Diseño (SOLID y DDD)

- Single Responsibility (SRP):** Cada módulo, componente, hook, servicio o utilidad tiene una sola razón para cambiar.
- Dependency Inversion (DIP):** La interfaz depende de contratos/abstracciones, nunca de implementaciones concretas de infraestructura.
- Domain-Driven Design (DDD):** El software utiliza el lenguaje ubicuo de la odontología (*hallazgoClínico*, *arancel*, *piezaAnatomica*, *proporcionAurea*, *cpod*).

CAPÍTULO II: ARQUITECTURA GENERAL Y ESTRUCTURA DE CARPETAS

2.1 Arquitectura por Capas

Flujo de dependencia unidireccional estricto (de afuera hacia adentro):

```
[ CAPA 1: PRESENTACIÓN (UI) ] —> React Components, JSX, Tailwind CSS
  |
  ▼
[ CAPA 2: APLICACIÓN (STATE) ] —> Custom Hooks, Context API
  |
  ▼
[ CAPA 3: DOMINIO (CORE) ] —> Reglas Odontológicas Puras, Esquemas, Validador CIE-10
  |
  ▼
[ CAPA 4: INFRAESTRUCTURA ] —> Repositorios LocalStorage / IndexedDB, Adaptadores API
```

2.2 Estructura General del Proyecto (`src/`)

```
src/
├─ assets/ # Recursos gráficos estáticos globales
├─ components/ # Componentes UI atómicos agnósticos al negocio (Button, Modal, Input)
├─ context/ # Estados globales del sistema (Auth, Theme, Notifications)
├─ data/ # Datos clínicos de referencia (Vademécum, CIE-10, Plantillas)
├─ hooks/ # Custom hooks globales reutilizables (useDebounce, useMedia)
├─ modules/ # Módulos de Dominio/Negocio
├─ services/ # Clientes de almacenamiento y comunicación externa
├─ utils/ # Formateadores puros y utilidades globales
├─ App.jsx # Orquestador Raíz
└─ main.jsx # Punto de Entrada React/Vite
```

2.3 Estructura Oficial de un Módulo (`src/modules/[nombreModulo]/`)

```
src/modules/[nombreModulo]/
├─ assets/ # SVGs o imágenes exclusivas del módulo
├─ components/ # Subcomponentes visuales del módulo
├─ constants/ # Enumeraciones, colores, valores por defecto
├─ hooks/ # Custom hooks de estado y ciclo de vida del módulo
├─ schemas/ # Validación de datos y estructuras (DTOs, contratos)
├─ services/ # Persistencia de datos local e integración I/O
├─ utils/ # Cálculos matemáticos y transformaciones de datos puras
├─ [NombreModulo]Modulo.jsx # Orquestador visual del módulo
└─ index.js # Pública API (Barrera de Exportación Estricta)
```

CAPÍTULO III: MATRIZ DE DECISIONES DE CÓDIGO (CUÁNDO CREAR QUÉ)

Algoritmo de Decisión de Arquitectura:

- ¿Elemento visual reutilizable sin lógica odontológica? → Componente Atómico (src/components/)
- ¿Cálculo matemático, transformación o algoritmo clínico puro? → Utilidad (modules/[modulo]/utils/)
- ¿Lectura/escritura de almacenamiento, red o APIs del navegador? → Servicio (modules/[modulo]/services/)
- ¿Estructura, valida o inicializa la forma de un objeto? → Esquema (modules/[modulo]/schemas/)
- ¿Maneja useState/useEffect o conecta UI con Services/Utils? → Custom Hook (modules/[modulo]/hooks/)

CAPÍTULO IV: REGLAS DE REFACTORIZACIÓN Y TAMAÑO LÍMITE DE ARCHIVOS

Tipo de Archivo	Límite Máximo	Acción Obligatoria al Superar Límite
Componentes JSX	250 líneas	Fragmentar en subcomponentes dentro de components/.
Custom Hooks	150 líneas	Aplicar <i>Hook Composition</i> (dividir en hooks especializados).
Funciones Utilitarias	50 líneas	Dividir en funciones puras atómicas de responsabilidad única.
Estado (useState)	5 declaraciones	Refactorizar a useReducer o estructurar un esquema.

CAPÍTULO V: REGLAS DE PERSISTENCIA HÍBRIDA Y SEGURIDAD CLÍNICA

5.1 Estrategia de Almacenamiento Local (Offline-First)

- **LocalStorage (Síncrono — Límite 5MB):** Reservado exclusivamente para configuraciones de usuario, banderas de estado, aranceles y sesiones.
- **IndexedDB (Asíncrono — Gran Capacidad):** Obligatorio para archivos binarios, fotografías DSD, radiografías y periodontogramas complejos.

5.2 Norma "Fail-Safe Clinical Default"

En cualquier cálculo de seguridad médica (alergias o dosis), si los datos del paciente están incompletos: **nunca se asumirá ausencia de riesgo**. La función devolverá el estado restrictivo: *"Datos Incompletos — Verificación Manual Requerida"*.

CAPÍTULO VI: PROTOCOLO DE DESARROLLO DE MÓDULOS Y DEFINITION OF DONE (DOD)

6.1 Secuencia Inviolable de Construcción (6 Pasos)

1. **Paso 1: Esquemas** (``schemas/``) — Estructura de datos pura y validación.
2. **Paso 2: Constantes** (``constants/``) — Enumeraciones, colores y claves.
3. **Paso 3: Algoritmos Puros** (``utils/``) — Cálculos de negocio sin dependencias React.
4. **Paso 4: Servicios y Hooks** (``services/``, ``hooks/``) — Persistencia local y reactividad.
5. **Paso 5: Ensamblaje Visual** (``components/``) — Interfaz de usuario expuesta.

6.2 Definition of Done (DoD)

- [x] Estructura estricta de subcarpetas respetada.
- [x] Encapsulamiento en `index.js` (Pública API).
- [x] Cero uso de `export default` en archivos internos (solo exportaciones nombradas).
- [x] Cumplimiento de límites de líneas y estados.
- [x] Manejo de impresiones mediante clases `print:hidden` / `print:block`.
- [x] Prueba de humo (Smoke Test) aprobada sin errores en consola.

CAPÍTULO VII: CHECKLIST OBLIGATORIO DE REVISIÓN PRE-COMMIT

- ✓ 1. Componentes en PascalCase, exportaciones nombradas.
- ✓ 2. Lógica matemática fuera de la vista (`.jsx`).
- ✓ 3. Comunicación inter-módulo por props/contratos vía `index.js`.
- ✓ 4. Bloques try/catch en toda llamada a LocalStorage/IndexedDB.
- ✓ 5. Aplicación de `React.memo` o `useMemo` en SVGs/listas extensas.

CAPÍTULO VIII: GOBERNANZA ARQUITECTÓNICA Y CONTROL DE CAMBIOS (RFC)

Para evitar modificaciones impulsivas, todo cambio mayor requerirá completar el protocolo RFC:

SOLICITUD DE CAMBIO ARQUITECTÓNICO (RFC)

1. ¿Qué problema clínico o técnico específico resuelve?
2. ¿Qué módulos o capas se verán afectados directa o indirectamente?
3. ¿Rompe la compatibilidad con el estado actual o la interfaz de algún módulo?
4. ¿Existe una solución más simple utilizando el código ya existente?
5. ¿Qué componentes, hooks, servicios o utilidades actuales se van a reutilizar?
6. ¿Afecta el rendimiento de renderizado o los límites de LocalStorage/IndexedDB?
7. ¿Cómo se probará la solución (smoke test, simulación clínica)?

CAPÍTULO IX: CONVENCIONES Y ESTÁNDARES DE CÓDIGO

9.1 Orden Obligatorio de Imports

```
// 1. React y Librerías Externas
import React, { useState, useMemo } from 'react';
import { LucideIcon } from 'lucide-react';

// 2. Componentes UI Atómicos / Contextos Globales
import { Modal } from '@components/ui/Modal';
import { useAuth } from '@context/AuthContext';

// 3. Módulos Externos (Vía Barrera Public API)
import { PresupuestosModulo } from '@modules/presupuestos';

// 4. Archivos Internos del Módulo
import { useOdontoAnatomico } from './hooks/useOdontoAnatomico';
import { calcularCPOD } from './utils/odontoCalculations';
import { ODONTO_CONSTANTS } from './constants/odontoConstants';
```

9.2 Documentación JSDoc Obligatoria

```
/**
 * Calcula el índice CPO-D de un paciente.
 * @param {Array} piezas - Arreglo de piezas dentales.
 * @returns {{ caridos: number, perdidos: number, obturados: number, totalCPOD: number }}
 */
export const calcularCPOD = (piezas = []) => { ... };
```

Algoritmo de trabajo en 10 pasos: **Análisis** → **Revisión Constitución** → **Búsqueda Reutilización** → **RFC** → **Aprobación**

Toda modificación requerirá un informe indicando: Archivos Creados, Modificados, Eliminados, Motivo, Riesgo

1. Pensar primero en la Arquitectura, después en el Código.
2. Priorizar Simplicidad sobre Complejidad Prematura (KISS).
3. Evitar la Duplicación (DYY).
4. Favorecer la Reutilización sobre la Creación.
5. Mantener el Desacoplamiento.
6. Diseñar pensando en 5 Años de Crecimiento.
7. Consistencia Estricta entre Módulos.
8. Justificación Técnica Obligatoria.

FASE 1: PROTOTIPO (Paz Arquitectónica)

Consolidación de la Constitución v3.0.0 y estandarización de la biblioteca de UI atómica.

FASE 2: BETA (Desfragmentación de Módulos)

Descomposición de FichaPaciente en submódulos e integración de validadores de esquemas.

FASE 3: PRODUCCIÓN (Estabilidad Local Multi-Especialidad)

Almacenamiento híbrido optimizado y motores de impresión PDF/reportes.

FASE 4: ESCALAMIENTO (Sincronización Cloud / Backend Hybrid)

Conectores de API remota manteniendo la operabilidad Offline-First.

FASE 5: PLATAFORMA EMPRESARIAL (Enterprise Clinical OS)

Integración con fichas de salud públicas, IA radiográfica y soporte multiclínica.

DECLARACIÓN OFICIAL DE VIGENCIA

La CONSTITUCIÓN DE ARQUITECTURA DE STUDIO DENTAL (v3.0.0) queda formalmente Aprobada, Consolidada y De